

CENTRO DE EDUCAÇÃO TECNOLÓGICA DO AMAZONAS – CETAM
ESCOLA DE EDUCAÇÃO PROFISSIONAL GALILÉIA
CURSO TÉCNICO DE NÍVEL MÉDIO EM DESENVOLVIMENTO DE SISTEMAS

**SISFEIRA: SISTEMA WEB DE GESTÃO DE PEDIDOS PARA FEIRA DE
PRODUTORES LOCAIS**

JOSÉ LUIZ
LUIS FERNANDO
PAULO VICTOR MONTEIRO

MANAUS – AM

2026

Ambiente de Aprendizagem onde foi realizada a Prática Profissional Supervisionada: ESCOLA DE EDUCAÇÃO PROFISSIONAL GALILÉIA

Endereço do local onde foi realizada a Prática Profissional Supervisionada: Endereço: Rua Marginal Esquerda, 28, Galileia, Manaus – AM (CEP 69093-780)

Ambiente de Aprendizagem: Laboratório de Informática 02

Turno: Matutino

Período de realização: 18/08/2026 à 08/09/2026

Relatório Técnico de Prática Profissional Supervisionada Nível II, apresentado como requisito parcial para aprovação no componente curricular do CURSO TÉCNICO DE NÍVEL MÉDIO EM DESENVOLVIMENTO DE SISTEMAS.

Nota Final das Prática Profissional Supervisionada da Etapa 01: () _____

Considerações do Instrutor da Prática Profissional Supervisionada:

Instrutora: Priscila Leylianne da Silva Gonçalves

CENTRO DE EDUCAÇÃO TECNOLÓGICA DO AMAZONAS – CETAM
ESCOLA DE EDUCAÇÃO PROFISSIONAL GALILÉIA
CURSO TÉCNICO DE NÍVEL MÉDIO EM DESENVOLVIMENTO DE SISTEMAS

Avaliação Final do Projeto Técnico:

A Avaliação do projeto intitulado (**SISFEIRA: SISTEMA WEB DE GESTÃO DE PEDIDOS PARA FEIRA DE PRODUTORES LOCAIS**), desenvolvido pelos estudantes: **José Luiz, Luis Fernando e Paulo Victor Monteiro**, está composta de duas partes, sendo:

Parte 1 – Avaliação do acompanhamento sistemático realizado no decorrer do desenvolvimento do projeto, obtendo a nota: ()_____

Parte 2 – Entrega do trabalho impresso e socialização da temática com a turma: ()_____

Nota final nesta Etapa do Componente (Projeto): ()_____

Considerações do Professor/Instrutor:

Instrutor do Projeto (Avaliador)

MANAUS – AM

2026

LISTA DE FIGURAS

Figura 1 – Diagrama de Casos de Uso do SISFEIRA	11
Figura 2 – Diagrama de Classes de Domínio	12
Figura 3 – Diagrama de Atividades e Ciclo de Vida do Pedido	13

LISTA DE TABELAS

Tabela 1 – Requisitos Funcionais (RF)	8
Tabela 2 – Requisitos Não Funcionais (RNF)	9
Tabela 3 – Regras de Negócio (RN)	10
Tabela 4 – Dicionário de Dados: Entidade Usuário	14
Tabela 5 – Dicionário de Dados: Entidade Produto	15
Tabela 6 – Dicionário de Dados: Entidade Edição da Feira	16
Tabela 7 – Dicionário de Dados: Entidade Ponto de Retirada	16
Tabela 8 – Dicionário de Dados: Entidade Pedido	17
Tabela 9 – Dicionário de Dados: Entidade Item do Pedido	18
Tabela 10 – Dicionário de Dados: Entidade Notificação	18
Tabela 11 – Fases de Execução do Projeto	23
Tabela 12 – Cronograma de Atividades	24
Tabela 13 – Orçamento Consolidado do Projeto	25

SUMÁRIO

Sumário

1 APRESENTAÇÃO

Este relatório apresenta a síntese das atividades desenvolvidas durante a etapa de Levantamento de Requisitos e Mapeamento de Processos referente à Prática Profissional Supervisionada no âmbito do Curso Técnico em Desenvolvimento de Sistemas do CETAM. Durante este período, as ações estiveram concentradas no estudo, especificação técnica e modelagem do projeto **SISFEIRA**, uma aplicação web voltada à modernização da cadeia de comercialização das feiras livres de produtores locais e agricultores familiares.

A execução desta prática justifica-se pela necessidade de alinhar o desenvolvimento de software aos preceitos da engenharia e da gestão de processos contemporânea. Ao permitir que os clientes visualizem a oferta semanal e realizem seus pedidos antecipadamente, o sistema possibilita que os feirantes colham e transportem estritamente a quantidade demandada, mitigando perdas financeiras e o descarte de alimentos perecíveis.

A solução técnica prioriza simplicidade operacional, controle e baixo custo, adotando tecnologias abertas (HTML5, CSS3, Vanilla JS, Node.js, Express e PostgreSQL) executadas nativamente em ambiente Debian Linux. Diante desse contexto, a presente etapa prática teve como objetivo geral mapear o fluxo operacional das feiras e estruturar a especificação técnica para a construção do Produto Mínimo Viável (MVP) em um ciclo de 15 dias. Como objetivos específicos, esta prática buscou:

- a) Mapear as rotinas de catálogo, reserva de itens e controle físico de entregas na banca;
- b) Identificar as falhas de previsão de demanda que afetam agricultores e consumidores;
- c) Aplicar ferramentas modernas de modelagem conceitual, prototipação e gerência de requisitos para fundamentar a viabilidade técnica do SISFEIRA.

2 CARACTERIZAÇÃO DO PROBLEMA E JUSTIFICATIVA

2.1 A Importância do Problema para a Comunidade

As feiras livres constituem o principal canal de escoamento da agricultura familiar no Amazonas. Contudo, a ausência de mecanismos digitais de previsão de demanda acarreta gargalos crônicos: desabastecimento precoce de mercadorias procuradas ou excedente de produtos perecíveis que acabam descartados. O SISFEIRA soluciona esse cenário ao criar uma vitrine de pedidos antecipados, proporcionando previsibilidade ao feirante e conveniência ao comprador.

2.2 Projetos Semelhantes e Complementaridade

Diferente de sistemas governamentais estaduais ou de grandes plataformas proprietárias de comércio eletrônico, o SISFEIRA atua de maneira descentralizada e leve. O sistema foca nas especificidades comunitárias locais, servindo como canal ágil de captação de encomendas sem a imposição de taxas transacionais ou burocracias de pagamento eletrônico no MVP.

2.3 Benefícios Esperados

- **Impacto Social:** Inclusão sociodigital dos feirantes por meio de interfaces responsivas (*Mobile-First*) de usabilidade direta.
- **Impacto Econômico:** Redução de perdas financeiras dos agricultores familiares com colheitas ajustadas à demanda real.
- **Impacto Ambiental:** Mitigação do desperdício de alimentos orgânicos e racionalização do frete das comunidades rurais até Manaus.

2.4 Embasamento na Literatura (Justificativa Técnica)

A digitalização de processos operacionais encontra sólido respaldo na literatura técnica. Segundo Pressman e Maxim (2016), a aplicação de requisitos rigorosos e modelagem conceitual assegura a sustentabilidade do software. Conforme Elmasri e Navathe (2018), a consistência transacional relacional (ACID) é indispensável para integridade de compras e estoques.

3 ÁREA DE ABRANGÊNCIA

A área de abrangência do projeto compreende o ambiente operacional das feiras livres e eventos de produtores locais no Estado do Amazonas, com foco inicial de aplicação em feiras comunitárias no município de Manaus. O sistema foi concebido com arquitetura genérica para atender diferentes espaços físicos.

Os agentes abrangidos pelo sistema dividem-se em três perfis:

- **Produtores Locais / Feirantes:** Cadastram seus itens agrícolas, acompanham notificações de compras e preparam as encomendas.
- **Clientes / Compradores:** Consultam o catálogo semanal e reservam previamente os produtos para retirada física.
- **Organização da Feira:** Coordena parâmetros operacionais, ativa edições semanais e define pontos de entrega/coleta.

4 OBJETIVOS

4.1 Objetivo Geral

Desenvolver e implementar uma aplicação web para a gestão de pedidos antecipados, catálogo de produtos locais e controle de pontos de retirada para feiras livres, estabelecendo um canal digital eficiente entre a agricultura familiar e os consumidores.

4.2 Objetivos Específicos

- Implementar o módulo de cadastro e gestão de produtores e seus respectivos catálogos de produtos;
- Desenvolver a interface pública de visualização do catálogo organizado por semana/edição da feira;
- Construir o carrinho de compras e o fluxo de emissão de pedidos sem exigência de pagamento eletrônico imediato;
- Disponibilizar o acompanhamento do status do pedido (RECEBIDO, EM_PREPARACAO e ENTREGUE) com emissão de comprovante digital com código único;
- Implementar protocolos de segurança e privacidade alinhados à LGPD;
- Gerar relatórios automatizados de vendas consolidadas exclusivos para cada produtor.

4.3 Delimitação do Escopo do MVP

- **Escopo Contemplado:** Cadastro de usuários, vitrine virtual, gestão de carrinho com persistência local, emissão de comprovante para retirada física, painel de controle do feirante e relatórios analíticos básicos.
- **Não-Escopo:** O sistema não contempla integração com gateways bancários/-cartão de crédito (o pagamento ocorre presencialmente na banca), rastreamento por GPS ou serviços de entrega terceirizada (delivery).

5 METAS

As metas do projeto representam as entregas parciais e finais quantificadas ao longo do período de 15 dias de execução:

- **Meta 1 (Dias 1 a 3):** Concluir 100% do levantamento e detalhamento dos Requisitos Funcionais e Não Funcionais no Documento de Especificação de Requisitos e estruturar o esquema do banco de dados relacional.
- **Meta 2 (Dias 4 a 6):** Elaborar e validar os protótipos navegáveis das 7 telas centrais da aplicação no Figma (Login/Cadastro, Catálogo, Carrinho, Comprovante, Painel do Produtor, Status do Pedido e Relatórios).
- **Meta 3 (Dias 7 a 11):** Desenvolver 100% dos módulos de back-end (API REST em Node.js com PostgreSQL) e as interfaces responsivas em Vanilla JS.
- **Meta 4 (Dias 12 a 13):** Executar os testes integrados de ponta a ponta simulando o fluxo completo de cadastro, montagem de pedido, comprovante e evolução de status.
- **Meta 5 (Dias 14 a 15):** Finalizar a documentação técnica consolidada, gerar o relatório impresso e estruturar a apresentação oral do projeto.

6 FASES DE EXECUÇÃO

6.1 Entrevistas (Questionário Google Forms)

Questionário de Diagnóstico: Processo de Matrícula

Pesquisa de Satisfação e Melhoria – Matrículas Escolares

Este questionário é rápido, anônimo e tem como objetivo entender as dificuldades do atual modelo de matrículas (presencial) para ajudar no desenvolvimento de uma nova solução tecnológica para a nossa comunidade.

1. Qual é o seu principal vínculo com a instituição de ensino? *(Tipo no Forms: Múltipla escolha)*

- () Sou aluno(a) do ensino médio ou de cursos técnicos
- () Sou pai/mãe ou responsável por um aluno
- () Sou funcionário(a) da secretaria ou setor administrativo
- () Sou professor(a) ou coordenador(a)

2. Como você avalia a sua experiência geral com o atual modelo físico (presencial) de matrículas e renovações na escola? *(Tipo no Forms: Escala linear de 1 a 5)*

- (1) Muito insatisfatória / Desgastante
- (2) Insatisfatória
- (3) Razoável / Indiferente
- (4) Satisfatória
- (5) Muito satisfatória / Excelente

3. No último período de matrículas, qual foi o tempo médio que você gastou presencialmente na escola (incluindo tempo de fila e atendimento)? *(Tipo no Forms: Múltipla escolha)*

- () Menos de 30 minutos
- () Entre 30 minutos e 1 hora
- () Entre 1 e 3 horas

- ☐ Mais de 3 horas
- ☐ Não participei do último processo de matrícula

4. Qual foi a maior dificuldade enfrentada durante o processo presencial? *(Tipo no Forms: Múltipla escolha)*

- ☐ Longas filas de espera e superlotação
- ☐ Falta de clareza sobre a quantidade de vagas disponíveis
- ☐ Burocracia com cópias de documentos físicos (papel)
- ☐ Dificuldade de ir à escola no horário de funcionamento
- ☐ Não tive nenhuma dificuldade

5. Pensando na sua rotina, qual é o seu nível de acesso à internet para utilizar um futuro sistema digital? *(Tipo no Forms: Múltipla escolha)*

- ☐ Tenho bom acesso à internet pelo celular e computador
- ☐ Tenho acesso à internet apenas pelo celular (smartphone)
- ☐ Meu acesso à internet é limitado ou instável
- ☐ Não tenho acesso à internet em casa

6. Você seria a favor da substituição do modelo presencial de matrículas por um sistema 100% online (SisMatrícula), que pudesse ser acessado de casa? *(Tipo no Forms: Múltipla escolha)*

- ☐ Sim, com certeza
- ☐ Sim, mas manteria o atendimento presencial apenas para quem não tem internet
- ☐ Não, prefiro o método tradicional no papel
- ☐ Sou indiferente

7. Você se sentiria seguro(a) enviando fotos ou PDFs dos seus documentos pessoais e comprovantes através de um sistema oficial da escola? *(Tipo no Forms: Múltipla escolha)*

- ☐ Sim, totalmente seguro(a)

- () Parcialmente, desde que o sistema garanta a proteção dos dados
- () Não me sentiria seguro(a)

8. Na sua opinião, qual deve ser a funcionalidade MAIS importante no novo sistema de matrículas? *(Tipo no Forms: Múltipla escolha)*

- () **Rapidez e estabilidade:** O sistema não travar no dia de abertura das vagas
- () **Transparência:** Mostrar em tempo real sua posição na fila ou as vagas restantes
- () **Facilidade de uso:** Ser simples e fácil de preencher pelo celular
- () **Comunicação:** Enviar alertas (por e-mail ou WhatsApp) confirmando que a matrícula deu certo

6.2 Requisitos e Regras de Negócio

Requisitos Funcionais (RF): O que o sistema deve fazer

Código	Identificador	Descrição do Requisito
RF01	Cadastro de Produtores e Produtos	Permitir o cadastro de produtores e o gerenciamento autônomo dos produtos de seu catálogo agrícola.
RF02	Cadastro de Clientes	Permitir o registro e a autenticação de clientes/compradores para emissão de pedidos.
RF03	Catálogo Semanal da Feira	Disponibilizar a visualização pública dos produtos cadastrados para a edição ativa da feira.
RF04	Carrinho de Compras	Permitir que o cliente selecione múltiplos itens, defina quantidades e estruture seu pedido.
RF05	Confirmação e Comprovante	Confirmar a compra, registrando o pedido e emitindo comprovante digital com código único.
RF06	Gestão do Status do Pedido	Permitir ao produtor acompanhar e atualizar o estado do pedido (RECEBIDO, EM_PREPARACAO, ENTREGUE).
RF07	Histórico de Pedidos	Disponibilizar ao cliente a consulta de compras anteriores e acompanhamento de pedidos ativos.
RF08	Relatório de Vendas	Gerar relatórios analíticos de vendas e faturamento consolidados exclusivos para cada produtor.
RF09	Notificação de Novos Pedidos	Emitir alertas no painel do produtor e disparar notificações transacionais de novas encomendas.
RF10	Definição do Ponto de Retirada	Exibir e exigir a seleção do local físico de coleta homologado para a feira.

Requisitos Não Funcionais (RNF): Restrições e qualidades do sistema

Código	Categoria	Descrição do Requisito
RNF01	Desempenho	Garantir carregamento das páginas de catálogo em tempo inferior a 2 segundos em rede padrão.
RNF02	Segurança	Proteger credenciais com hash seguro (<i>bcrypt</i>) e parametrizar queries SQL (\$1, \$2).
RNF03	Usabilidade	Fornecer fluxo de compra simples e intuitivo, realizável em poucos cliques.
RNF04	Disponibilidade	Assegurar disponibilidade estável do servidor durante as janelas de maior demanda da feira.
RNF05	Portabilidade	Estruturar layout responsivo otimizado para smartphones (<i>Mobile-First</i>).
RNF06	Escalabilidade	Suportar acessos simultâneos sem esgotar conexões via <i>Connection Pooling</i> no PostgreSQL.
RNF07	Manutenibilidade	Adotar arquitetura modular desacoplada separando rotas, controladores, serviços e repositórios.
RNF08	Integridade Transacional	Garantir consistência estrita (ACID) na persistência de pedidos via transações atômicas nativas.

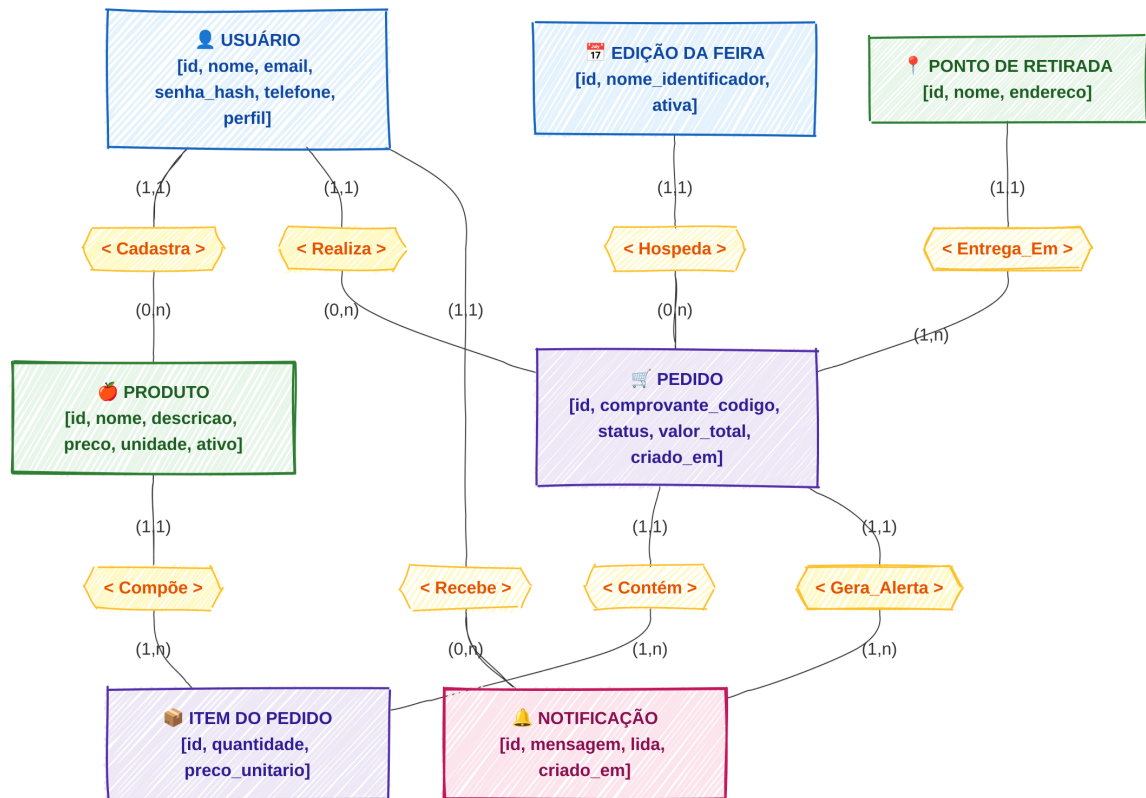
Regras de Negócio (RN): Condições e políticas que o sistema deve respeitar

Código	Dimensão / Foco	Enunciado da Regra de Negócio
RNG/SEG 01	Segurança e Sessão	A autenticação de usuários opera via tokens JWT (expiração padrão de 8h) com autorização RBAC no back-end.
RNG/SEG 02	Isolamento de Dados	Consultas analíticas e relatórios devem filtrar estritamente o <code>produtor_id</code> extraído do token.
RNG/USA 03	Usabilidade e Contato	A confirmação do pedido gera código identificador único e link dinâmico formatado para WhatsApp do feirante.
RNG/USA 04	Ciclo de Atendimento	O status do pedido evolui exclusivamente de forma sequencial: RECEBIDO → EM_PREPARACAO → ENTREGUE.
RNG/DES 05	Desempenho e Leveza	A interface gráfica adota arquitetura Multi-Page Application (MPA) em Vanilla JS sem compilação (<i>Zero-Build</i>).
RNG/DES 06	Notificação Assíncrona	A criação do pedido registra o alerta no banco e despacha a notificação de e-mail em segundo plano.
RNG/CUS 07	Custo Zero de Licença	Utilização exclusiva de tecnologias abertas e gratuitas (Node.js, PostgreSQL, Debian Linux).
RNG/CUS 08	Infraestrutura Host Único	Execução direta em modo <i>Bare-Metal</i> no Linux dispensando despesas com servidores proprietários.

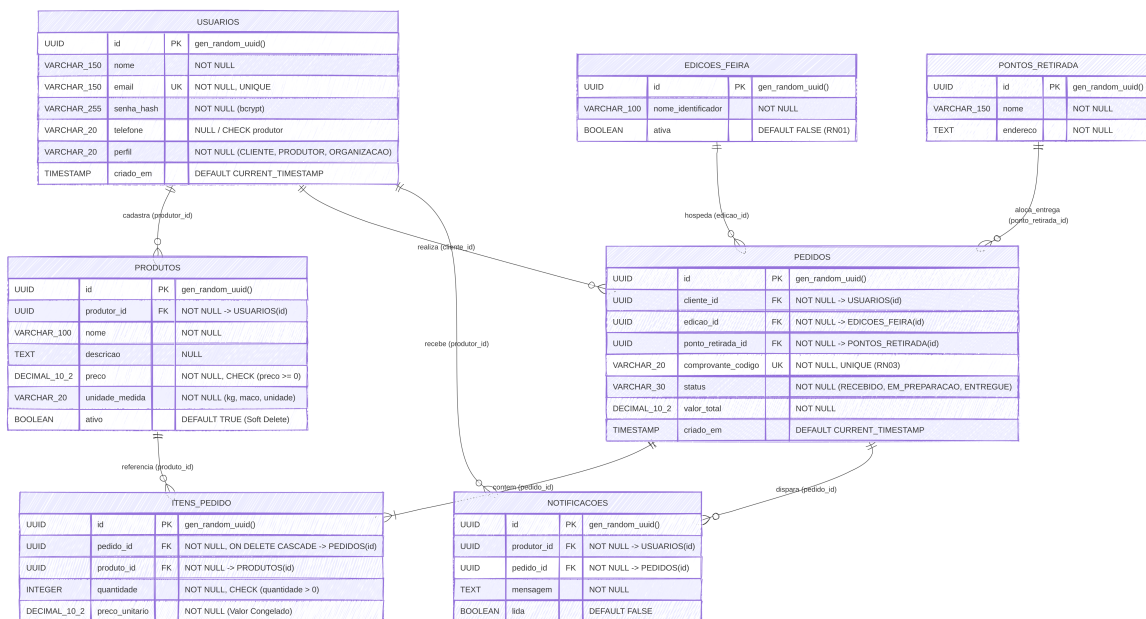
6.3 Modelagem (Banco de Dados)

Entidades e Atributos: Definição formal das entidades do sistema, atributos tipados e cardinalidade dos relacionamentos que compõem o banco de dados relacional.

Modelo Conceitual: Diagrama de alto nível representando entidades, atributos e associações.



Modelo Lógico: Estruturação das entidades em formato de tabelas com chaves primárias e estrangeiras.



Modelo Físico: Scripts e comandos DDL em código SQL para criação do banco de dados relacional.

6.4 Script de Criação do Banco de Dados

O script apresentado a seguir demonstra a estrutura inicial do banco de dados do SIS-FEIRA, incluindo a criação das tabelas, restrições de integridade, chaves estrangeiras e índices utilizados pela aplicação.

```
1  -- ARQUIVO 1: src/database/migrations/01_schema_inicial.sql
2
3  CREATE TABLE usuarios (
4      id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
5      nome VARCHAR(150) NOT NULL,
6      email VARCHAR(150) NOT NULL UNIQUE,
7      senha_hash VARCHAR(255) NOT NULL,
8      telefone VARCHAR(20) NULL,
9      perfil VARCHAR(20) NOT NULL,
10     criado_em TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
11
12     CONSTRAINT chk_telefone_produto
13         CHECK (perfil != 'PRODUTOR' OR telefone IS NOT NULL),
14
15     CONSTRAINT chk_perfil_valido
16         CHECK (perfil IN ('CLIENTE', 'PRODUTOR', 'ORGANIZACAO'))
17 );
18
19 CREATE TABLE produtos (
20     id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
21     produtor_id UUID NOT NULL REFERENCES usuarios(id),
22     nome VARCHAR(100) NOT NULL,
23     descricao TEXT NULL,
24     preco DECIMAL(10,2) NOT NULL,
25     unidade_medida VARCHAR(20) NOT NULL,
26     ativo BOOLEAN DEFAULT TRUE,
27
28     CONSTRAINT chk_preco_positivo
29         CHECK (preco >= 0)
30 );
```

```

31
32 CREATE TABLE edicoes_feira (
33     id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
34     nome_identificador VARCHAR(100) NOT NULL,
35     ativa BOOLEAN DEFAULT FALSE
36 );
37
38 CREATE TABLE pontos_retirada (
39     id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
40     nome VARCHAR(150) NOT NULL,
41     endereco TEXT NOT NULL
42 );
43
44 CREATE TABLE pedidos (
45     id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
46     cliente_id UUID NOT NULL REFERENCES usuarios(id),
47     edicao_id UUID NOT NULL REFERENCES edicoes_feira(id),
48     ponto_retirada_id UUID NOT NULL REFERENCES pontos_retirada(id),
49     comprovante_codigo VARCHAR(20) NOT NULL UNIQUE,
50     status VARCHAR(30) NOT NULL,
51     valor_total DECIMAL(10,2) NOT NULL,
52     criado_em TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
53
54     CONSTRAINT chk_status_pedido
55         CHECK (
56             status IN (
57                 'RECEBIDO',
58                 'EM_PREPARACAO',
59                 'ENTREGUE'
60             )
61         )
62 );
63
64 CREATE TABLE itens_pedido (

```



```

65     id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
66     pedido_id UUID NOT NULL
67         REFERENCES pedidos(id) ON DELETE CASCADE,
68     produto_id UUID NOT NULL REFERENCES produtos(id),
69     quantidade INTEGER NOT NULL,
70     preco_unitario DECIMAL(10,2) NOT NULL,
71
72     CONSTRAINT chk_quantidade_positiva
73         CHECK (quantidade > 0)
74 );
75
76 CREATE TABLE notificacoes (
77     id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
78     produtor_id UUID NOT NULL REFERENCES usuarios(id),
79     pedido_id UUID NOT NULL REFERENCES pedidos(id),
80     mensagem TEXT NOT NULL,
81     lida BOOLEAN DEFAULT FALSE
82 );
83
84 CREATE INDEX idx_produtos_produtor
85     ON produtos(produtor_id);
86
87 CREATE INDEX idx_pedidos_cliente
88     ON pedidos(cliente_id, criado_em DESC);
89
90 CREATE INDEX idx_notificacoes_nao_lidas
91     ON notificacoes(produtor_id)
92     WHERE lida = FALSE;
93
94 CREATE INDEX idx_itens_pedido_agrupamento
95     ON itens_pedido(produto_id);

```

Listing 1: Script SQL de criação do banco de dados

7 METODOLOGIA

A metodologia adotada para o desenvolvimento do SISFEIRA fundamenta-se nas práticas da Engenharia de Software para entrega de um Produto Mínimo Viável (MVP), aliando modelagem conceitual UML, análise de fluxos e definição estrita do Dicionário de Dados.

7.1 Diagrama de Casos de Uso

Representação formal dos atores mapeados (Cliente, Produtor Local, Organização da Feira e Serviço de Notificações) e das ações disponibilizadas pela aplicação.

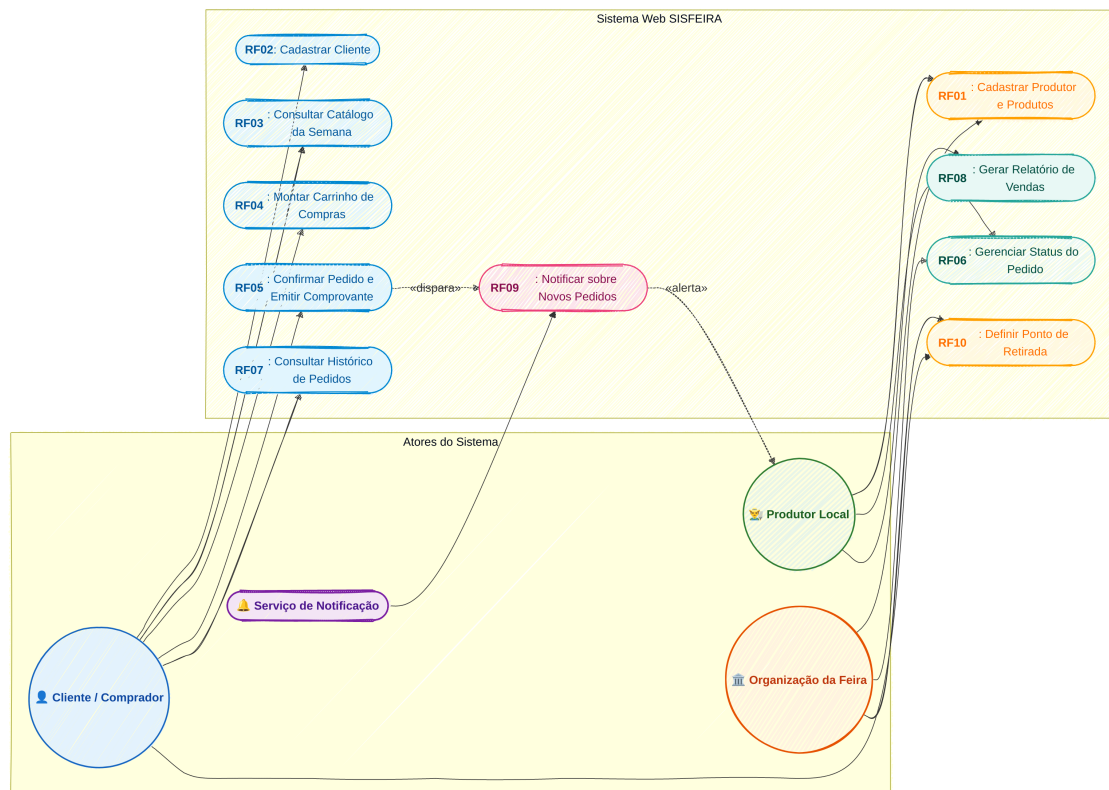


Figura 1 – Diagrama de Casos de Uso do SISFEIRA

7.2 Diagrama de Classes

Estrutura orientada a objetos da camada de domínio contendo nomes de classes, atributos tipados e cardinalidade dos relacionamentos.

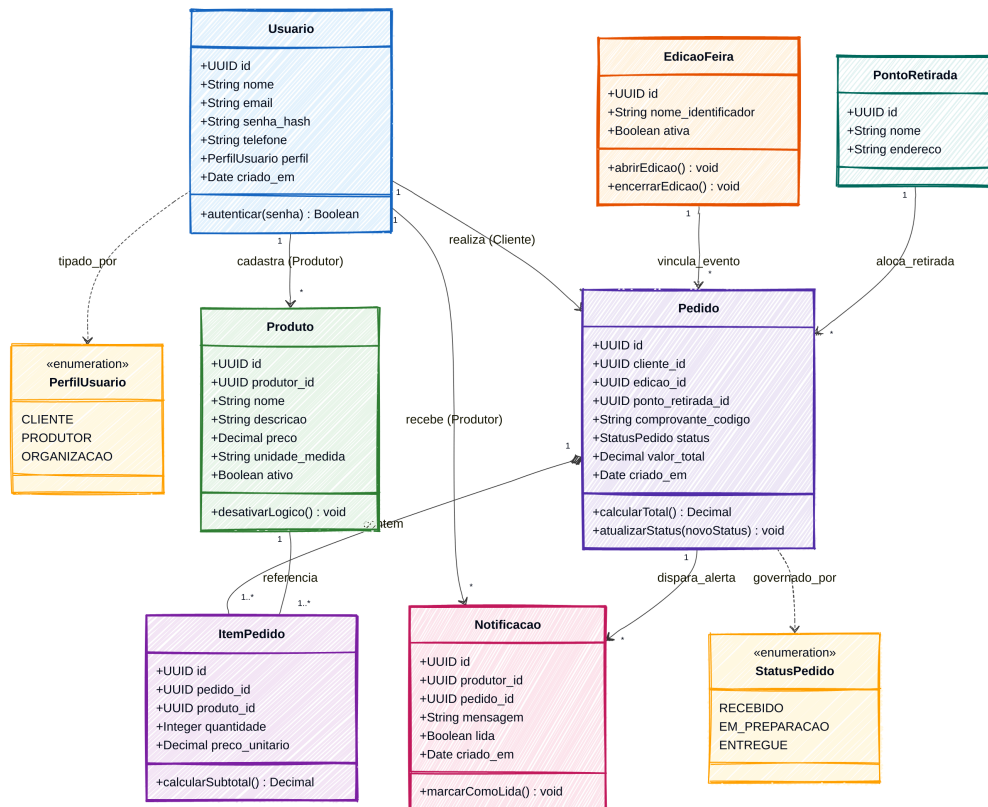


Figura 2 – Diagrama de Classes de Domínio

7.3 Diagrama de Atividades

Representação do fluxo de processos e ciclo de vida da jornada do cliente e do produtor, da montagem do carrinho à retirada na feira.

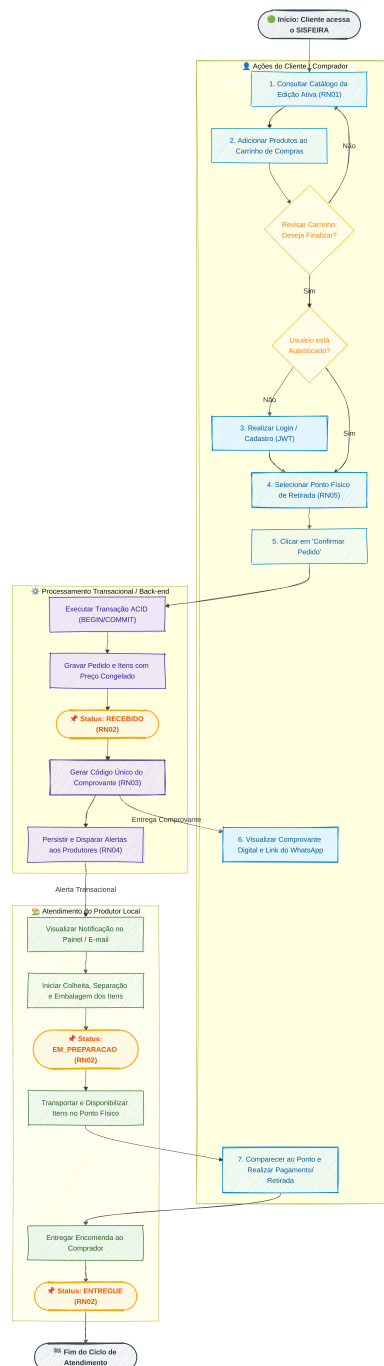


Figura 3 – Diagrama de Atividades e Ciclo de Vida do Pedido

7.4 Dicionário de Dados (DD)

O Dicionário de Dados formaliza o esquema relacional no PostgreSQL, definindo metadados, volume esperado, regras de retenção e restrições de cada tabela.

ENTIDADE: Usuário	
Descrição:	Cadastro de clientes, produtores e organizadores da feira.
Nome da Tabela:	<code>usuarios</code>
Volume Esperado:	≈ 100 a 500 registros ativos na fase piloto.
Tempo de Retenção:	Permanente enquanto a conta mantiver-se ativa.
Rotina de Limpeza:	Inativação lógica conforme solicitação do titular (LGPD).

ATRIBUTOS: usuarios					
Atributo	Nome Campo	Tipo	Tam.	Restrição	Descrição
Identificador	<code>id</code>	UUID	36	PK, NOT NULL	Identificador único gerado por <code>gen_random_uuid()</code> .
Nome	<code>nome</code>	VARCHAR	50	NOT NULL	Nome civil completo ou razão social do feirante.
E-mail	<code>email</code>	VARCHAR	50	NOT NULL, UK	Login único utilizado para emissão do JWT.
Hash Senha	<code>senha_hash</code>	VARCHAR	255	NOT NULL	Hash seguro criptografado via <i>bcrypt</i> (RNF02).
Telefone	<code>telefone</code>	VARCHAR	20	NULL / CHECK	Contato com DDD; obrigatório para perfil PRODUTOR.
Perfil	<code>perfil</code>	VARCHAR	20	NOT NULL, CHECK	Papel de acesso: 'CLIENTE', 'PRODUTOR' ou 'ORGANIZACAO'.
Criação	<code>criado_em</code>	TIMESTAMP		DEFAULT NOW()	Data e hora de criação do registro no banco.

ENTIDADE: Produto	
Descrição:	Catálogo de itens agrícolas e artesanais comercializados.
Nome da Tabela:	produtos
Volume Esperado:	≈ 200 a 1.000 itens por edição de feira.
Tempo de Retenção:	Permanente para preservação do histórico de compras.
Rotina de Limpeza:	Desativação lógica (<i>Soft Delete</i> , ativo = FALSE).

ATRIBUTOS: produtos					
Atributo	Nome Campo	Tipo	Tam.	Restrição	Descrição
Identificador	id	UUID	36	PK, NOT NULL	Identificador universal único do produto.
Produtor	produtor_id	UUID	36	FK, NOT NULL	Vínculo com usuarios(id) para isolamento de itens.
Nome	nome	VARCHAR	100	NOT NULL	Denominação comercial do item agrícola ofertado.
Descrição	descricao	TEXT	–	NULL	Detalhes sobre cultivo, apresentação e características.
Preço	preco	DECIMAL	10,2	NOT NULL, CHECK	Valor unitário oficial de venda (≥ 0).
Unidade	unidade_medida	VARCHAR	20	NOT NULL	Unidade padrão de venda (ex.: 'kg', 'maço').
Ativo	ativo	BOOLEAN	1	DEFAULT TRUE	Flag que controla a visibilidade no catálogo semanal.

ENTIDADE: Edição da Feira	
Descrição:	Ciclos temporais de realização das feiras presenciais.
Nome da Tabela:	edicoes_feira
Volume Esperado:	≈ 50 registros por ano de operação.
Tempo de Retenção:	5 anos para histórico e auditoria.
Rotina de Limpeza:	Arquivamento anual de edições encerradas.

ATRIBUTOS: edicoes_feira					
Atributo	Nome Campo	Tipo	Tam.	Restrição	Descrição
Identificador	id	UUID	36	PK, NOT NULL	Identificador universal da edição.
Nome Edição	nome_identificacao	VARCHAR	100	NOT NULL	Rótulo descritivo do evento (ex.: 'Semana 42 - 2026').
Ativa	ativa	BOOLEAN	1	DEFAULT FALSE	Define se a feira está aberta para pedidos (RN01).

ENTIDADE: Ponto de Retirada	
Descrição:	Locais físicos para recolhimento presencial das encomendas.
Nome da Tabela:	pontos_retirada
Volume Esperado:	≈ 5 a 20 pontos cadastrados.
Tempo de Retenção:	Permanente.
Rotina de Limpeza:	Manutenção manual conforme logística da feira.

ATRIBUTOS: pontos_retirada					
Atributo	Nome Campo	Tipo	Tam.	Restrição	Descrição
Identificador	id	UUID	36	PK, NOT NULL	Identificador universal do ponto de logística.
Nome	nome	VARCHAR	50	NOT NULL	Nome do ponto físico (ex.: 'Tenda Principal').
Endereço	endereco	TEXT	–	NOT NULL	Logradouro completo com referências de retirada.

ENTIDADE: Pedido	
Descrição:	Intenções de compra confirmadas pelos clientes na feira.
Nome da Tabela:	pedidos
Volume Esperado:	≈ 500 a 5.000 pedidos por mês.
Tempo de Retenção:	2 anos para histórico e conciliação de vendas.
Rotina de Limpeza:	Consolidação periódica em base analítica.

ATRIBUTOS: pedidos					
Atributo	Nome Campo	Tipo	Tam.	Restrição	Descrição
Identificador	id	UUID	36	PK, NOT NULL	Identificador universal do pedido.
Cliente	cliente_id	UUID	36	FK, NOT NULL	Vínculo com o comprador em usuarios(id).
Edição	edicao_id	UUID	36	FK, NOT NULL	Vínculo com o evento em edicoes_feira(id).
Ponto Retirada	ponto_retirada_id	UUID	36	FK, NOT NULL	Vínculo com o local em pontos_retirada(id).
Comprovante	comprovante_codigo	VARCHAR	20	NOT NULL, UK	Código único gerado para conferência (RN03).
Status	status	VARCHAR	30	NOT NULL, CHECK	'RECEBIDO', 'EM_PREPARACAO' ou 'ENTREGUE'.
Valor Total	valor_total	DECIMAL	10,2	NOT NULL	Montante financeiro total calculado no back-end.
Criação	criado_em	TIMESTAMP		DEFAULT NOW()	Data e hora do fechamento da solicitação.

ENTIDADE: Item do Pedido	
Descrição:	Detalhamento dos produtos e valores congelados na compra.
Nome da Tabela:	itens_pedido
Volume Esperado:	≈ 2.000 a 20.000 itens processados mensalmente.
Tempo de Retenção:	2 anos, estritamente atrelado ao pedido pai.
Rotina de Limpeza:	Exclusão em cascata (<i>ON DELETE CASCADE</i>).

ATRIBUTOS: itens_pedido					
Atributo	Nome Campo	Tipo	Tam.	Restrição	Descrição
Identificador	id	UUID	36	PK, NOT NULL	Identificador universal do item do pedido.
Pedido	pedido_id	UUID	36	FK, NOT NULL	Vínculo com o pedido pai em pedidos(id).
Produto	produto_id	UUID	36	FK, NOT NULL	Vínculo com o catálogo em produtos(id).
Quantidade	quantidade	INTEGER	4	NOT NULL, CHECK	Volume demandado pelo comprador (> 0).
Preço Unitário	preco_unitario	DECIMAL	10,2	NOT NULL	Preço unitário congelado no fechamento da compra.

ENTIDADE: Notificação	
Descrição:	Alertas internos aos feirantes sobre novos pedidos.
Nome da Tabela:	notificacoes
Volume Esperado:	≈ 1.000 a 5.000 alertas gerados por mês.
Tempo de Retenção:	6 meses a partir da data de leitura.
Rotina de Limpeza:	Expurgo periódico de alertas lidos com mais de 90 dias.

ATRIBUTOS: notificacoes					
Atributo	Nome Campo	Tipo	Tam.	Restrição	Descrição
Identificador	id	UUID	36	PK, NOT NULL	Identificador universal do alerta.
Produtor	produtor_id	UUID	36	FK, NOT NULL	Vínculo com o feirante em usuarios(id) (RN04).
Pedido	pedido_id	UUID	36	FK, NOT NULL	Vínculo com a encomenda em pedidos(id).
Mensagem	mensagem	TEXT	–	NOT NULL	Resumo dos itens e quantidades encomendadas.
Lida	lida	BOOLEAN	1	DEFAULT FALSE	Flag de controle de pendências no painel.

8 DESCRIÇÃO DAS ATIVIDADES PRÁTICAS NA ETAPA

O desenvolvimento do projeto apoiou-se em um conjunto de ferramentas integradas cobrindo todo o ciclo de vida do software:

8.1 Trello

O Trello é uma ferramenta para a execução de projetos baseada no método Kanban. Seu layout é formado por quadros que possibilitam inserir cartões com informações sobre as tarefas a realizar, em andamento, em revisão e concluídas. Permite controle rigoroso do prazo de entrega e visualização integrada do backlog de funcionalidades (FEAT-01 a FEAT-07).

8.2 Figma

Plataforma de design vetorial baseada em nuvem utilizada para a concepção de interfaces de usuário (UI), experiência do usuário (UX) e prototipagem colaborativa. Permite a construção de layouts responsivos otimizados para smartphones (*Mobile-First*), simulando os fluxos de catálogo, carrinho, checkout e relatórios.

8.3 Astah / Mermaid.js

Ferramentas consolidadas para modelagem visual e representação de diagramas sob o padrão UML (Casos de Uso, Classes de Domínio e Atividades) e Modelo Entidade-Relacionamento (DER), garantindo padronização e documentação clara dos artefatos conceituais.

8.4 Git e GitHub

O Git é um sistema de controle de versão distribuído utilizado no terminal Linux para rastrear o histórico de modificações atômicas no código-fonte (*commits* semânticos e ramificações por *feature*). O GitHub atuou como repositório remoto centralizado para armazenamento e colaboração entre os desenvolvedores.

8.5 PostgreSQL e Utilitário `psql`

SGBD relacional de alto desempenho executado nativamente em ambiente Debian Linux na porta 5432. Utilizou-se o utilitário de terminal `psql` para execução direta das migrações de esquema (*migrations*) e inserção dos dados de teste (*seeds*) sem dependência de intermediadores gráficos pesados.

9 ÓRGÃOS ENVOLVIDOS

A execução do presente projeto técnico envolve exclusivamente as seguintes entidades e agentes:

- **Centro de Educação Tecnológica do Amazonas – CETAM:** Entidade pública estadual de educação profissional, responsável pelo suporte institucional, supervisão técnico-pedagógica através da ESCOLA DE EDUCAÇÃO PROFISSIONAL GALILÉIA e avaliação curricular.
- **Equipe de Estudantes Desenvolvedores:** Formada pelos alunos José Luiz, Luis Fernando e Paulo Victor Monteiro, responsáveis pelo levantamento de requisitos, modelagem de dados, desenvolvimento do código-fonte, testes integrados e documentação técnica.

10 MECANISMOS E NORMAS DE EXECUÇÃO

10.1 Privacidade e Proteção de Dados (LGPD)

Em alinhamento à Lei Geral de Proteção de Dados Pessoais (LGPD - Lei nº 13.709/2018), o sistema implementa o princípio da coleta mínima: solicitam-se apenas nome, e-mail para autenticação e telefone para contato via WhatsApp. Não há captura nem retenção de dados bancários ou documentos sensíveis.

10.2 Segurança e Simplificações do MVP

Para a versão de MVP executada em ambiente de testes local no Debian Linux, foram adotadas decisões técnicas proporcionais:

- As senhas são protegidas com o algoritmo criptográfico *bcrypt* e as consultas SQL são blindadas contra injeção de código via parâmetros posicionais (\$1, \$2) no driver *pg*.
- **Armazenamento de JWT em `localStorage` (Risco Aceito):** Facilita o controle de sessão sem ferramentas de compilação (*Zero-Build*). Em ambiente público de produção com movimentação financeira, recomenda-se a migração para cookies protegidos com as flags `HttpOnly` e `Secure`.
- **Execução em Host Único Bare-Metal (Risco Aceito):** O servidor Node.js e o PostgreSQL rodam nativamente na mesma máquina comunicando-se via `loopback local`. Essa decisão elimina o overhead de orquestradores de contêineres, sendo adequada para a avaliação acadêmica.

11 ORÇAMENTO

Pela utilização exclusiva de tecnologias abertas, software livre e execução em ambiente computacional pré-existente dos estudantes, os custos diretos limitam-se a despesas de material de consumo para apresentação acadêmica:

Nº	Descrição	Unid.	Qtd.	V. Unit. (R\$)	Total (R\$)
Material de Expediente e Impressão					
1	Impressão da Documentação Técnica Final	Folha	30	0,50	15,00
2	Encadernação em Espiral com Capa Plástica	Unid.	1	10,00	10,00
3	Papel Sulfite A4 para Rascunhos e Validação	Pacote	1	10,00	10,00
Subtotal Material:					35,00
Infraestrutura e Licenças de Software					
1	Licenças de Software (Node.js, PostgreSQL, VS Code)	–	–	0,00	0,00
2	Ambiente de Hospedagem (Host Único Debian Linux)	–	–	0,00	0,00
Subtotal Software:					0,00
Valor Total do Projeto (R\$):					35,00

12 CRONOGRAMA

12.1 Fases de Execução

A distribuição temporal do projeto ocorreu em 5 blocos sequenciais de 3 dias:

Fases de Execução	D 1-3	D 4-6	D 7-9	D 10-12	D 13-15
Fase 1: Levantamento de Requisitos e Modelagem DER	X				
Fase 2: Prototipação de Interfaces e Planejamento		X			
Fase 3: Implementação do Back-end e Banco de Dados			X		
Fase 4: Implementação do Front-end e Integração				X	
Fase 5: Testes Integrados, Documentação e Apresentação					X

12.2 Cronograma de Atividades

O cronograma operacional detalha as atividades desenvolvidas ao longo das duas semanas de trabalho:

Atividades Previstas	D 1-3	D 4-6	D 7-9	D 10-12	D 13-15
Levantamento de Requisitos e Elaboração do DER	X				
Modelagem Entidade-Relacionamento do Banco	X				
Criação dos Protótipos de Interface (Figma)		X			
Configuração do Ambiente e Estruturação do Banco		X			
Desenvolvimento das Rotas e API (Node.js/Express)			X		
Desenvolvimento do Front-end (HTML5/CSS3/JS)				X	
Integração Front-end / Back-end e Validação				X	
Bateria de Testes Funcionais do MVP					X
Consolidação do Relatório Técnico e Apresentação					X

13 CONSIDERAÇÕES FINAIS

O desenvolvimento do projeto SISFEIRA permitiu consolidar, ao longo da Prática Profissional Supervisionada, uma especificação técnica estruturada para uma aplicação web destinada à gestão de pedidos antecipados, catálogo de produtos e pontos de retirada em feiras de produtores locais. A partir do levantamento do problema, da definição dos requisitos, da modelagem dos dados e dos processos e da organização das atividades de desenvolvimento, foi possível estabelecer uma base técnica coerente para a construção do Produto Mínimo Viável proposto.

A solução foi concebida considerando as características do contexto das feiras locais, priorizando simplicidade operacional, baixo custo de implementação, acesso por dispositivos móveis e redução da complexidade tecnológica. A definição de funcionalidades como catálogo semanal, carrinho de compras, emissão de comprovante, acompanhamento do pedido e notificações busca aproximar a oferta dos produtores da demanda dos consumidores, mantendo a retirada física e o pagamento presencial como elementos do escopo inicial.

Do ponto de vista técnico, a atividade também possibilitou a aplicação prática de conceitos de Engenharia de Software, incluindo levantamento e especificação de requisitos, modelagem UML, modelagem de banco de dados relacional, definição de regras de negócio, organização do ciclo de desenvolvimento e utilização de ferramentas de versionamento e prototipação. A adoção de tecnologias abertas e de uma arquitetura compatível com a infraestrutura disponível contribuiu para manter o projeto adequado às restrições de custo e ao caráter acadêmico da proposta.

Por se tratar de um MVP desenvolvido no contexto da prática profissional, algumas funcionalidades e integrações permanecem deliberadamente fora do escopo, como gateways de pagamento, serviços de entrega e outras infraestruturas externas. Dessa forma, a documentação produzida deve ser compreendida como a base técnica para uma implementação inicial e para futuras evoluções do sistema, permitindo que novas necessidades sejam incorporadas sem comprometer a estrutura fundamental definida nesta etapa.

Conclui-se, portanto, que o SISFEIRA apresenta uma proposta tecnicamente estruturada e compatível com o problema identificado, reunindo os elementos necessários para orientar sua implementação e posterior validação em ambiente real. A

etapa realizada contribuiu não apenas para a definição do sistema, mas também para o desenvolvimento das competências práticas relacionadas à análise, modelagem, planejamento e documentação de soluções de software.

14 REFERÊNCIAS

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 6023**: Informação e documentação – Referências – Elaboração. Rio de Janeiro: ABNT, 2018.

BRASIL. **Lei nº 13.709, de 14 de agosto de 2018**. Lei Geral de Proteção de Dados Pessoais (LGPD). Brasília, DF: Presidência da República, 2018.

CENTRO DE EDUCAÇÃO TECNOLÓGICA DO AMAZONAS (CETAM). **Orientações Metodológicas para Elaboração de Relatórios Técnicos e Prática Profissional**. Manaus: CETAM, 2026.

ELMASRI, Ramez; NAVATHE, Shamkant B. **Sistemas de Banco de Dados**. 7. ed. São Paulo: Pearson, 2018.

PRESSMAN, Roger S.; MAXIM, Bruce R. **Engenharia de Software**: uma abordagem profissional. 8. ed. Porto Alegre: AMGH, 2016.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. **PostgreSQL 15 Documentation**. 2023. Disponível em: <<https://www.postgresql.org/docs/>>. Acesso em: 15 fev. 2026.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 14724**: Informação e documentação – Trabalhos acadêmicos – Apresentação. 3. ed. Rio de Janeiro: ABNT, 2011.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 6023**: Informação e documentação – Referências – Elaboração. Rio de Janeiro: ABNT, 2018.

BRASIL. Conselho Nacional de Educação. Conselho Pleno. **Resolução CNE/CP nº 1, de 5 de janeiro de 2021**. Define as Diretrizes Curriculares Nacionais Gerais para a Educação Profissional e Tecnológica. Brasília, DF: MEC/CNE, 2021. Disponível em: <<https://www.in.gov.br/web/dou/-/resolucao-cne/cp-n-1-de-5-de-janeiro-de-2021-297767578>>. Acesso em: 27 jun. 2023.

BRASIL. Departamento Nacional de Produção Mineral (DNPM). **Metodologia de Desenvolvimento de Sistemas**. Versão 1.0. Brasília, DF: DNPM, 2008.

BRASIL. Ministério da Educação. **Catálogo Nacional de Cursos Técnicos (CNCT)**. 4. ed. Brasília, DF: MEC/Secretaria de Educação Profissional e Tecnológica, 2021. Disponível em: <<http://cnct.mec.gov.br/cnct-api/catalogopdf>>. Acesso em: 27 jun. 2023.

BRASIL. **Lei nº 13.709, de 14 de agosto de 2018**. Lei Geral de Proteção de Dados Pessoais (LGPD). Brasília, DF: Presidência da República, 2018.

BRASSCOM (Associação Brasileira das Empresas de Tecnologia da Informação e Comunicação). **Demanda de Talentos em TIC e Estratégia TCEM**. São Paulo: Brasscom, 2021. Disponível em: <<https://brasscom.org.br/pdfs/demanda-de-talentos-em-tic-e-estrategia-tcem/>>. Acesso em: 27 jun. 2023.

CENTRO DE EDUCAÇÃO TECNOLÓGICA DO AMAZONAS (CETAM). **Orientações Metodológicas para Elaboração de Relatórios Técnicos e Prática Profissional**. Manaus: CETAM, 2026.

ELMASRI, Ramez; NAVATHE, Shamkant B. **Sistemas de Banco de Dados**. 7. ed. São Paulo: Pearson, 2018.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. **PostgreSQL 15 Documentation**. 2023. Disponível em: <<https://www.postgresql.org/docs/>>. Acesso em: 15 fev. 2026.

PRESSMAN, Roger S.; MAXIM, Bruce R. **Engenharia de Software**: uma abordagem profissional. 8. ed. Porto Alegre: AMGH, 2016.

SOUZA, Ludmilla. Indústria de Software e Serviços de TIC cresceu 6,5% em 2021. **Agência Brasil**, São Paulo, 19 jul. 2022. Disponível em: <<https://agenciabrasil.ebc.com.br/economia/07/industria-de-software-e-servicos-de-tic-cresceram-65-em-2021>>. Acesso em: 27 jun. 2023.

ANEXO

Código-Fonte do Projeto

O código-fonte apresentado neste trabalho corresponde à implementação do sistema SISFEIRA desenvolvida durante a execução do projeto...

Arquivo: src/server.js

```
1 // src/server.js
2 require('dotenv').config();
3 const express = require('express');
4 const cors = require('cors');
5 const path = require('path');
6 const pool = require('./config/db');
7 const errorHandler = require('./middlewares/error.middleware');
8 const asyncHandler = require('./middlewares/async.middleware');
9 const authRoutes = require('./modules/auth/auth.routes');
10 const catalogRoutes = require('./modules/catalog/catalog.routes');
11 const orderRoutes = require('./modules/order/order.routes');
12 const notificationRoutes = require('./modules/notification/
    notification.routes');
13 const reportRoutes = require('./modules/report/report.routes');
14
15 const app = express();
16
17 app.use(cors());
18 app.use(express.json());
19 app.use(express.static(path.join(__dirname, '../frontend')));
20
21 app.use((req, res, next) => {
22     console.log(`${req.method} ${req.url}`);
23     next();
24 });
25
26 app.get('/api/health', asyncHandler(async (req, res) => {
```

```
27   await pool.query('SELECT 1');
28   res.status(200).json({ status: 'UP', database: 'connected' });
29 });
30
31
32 app.use('/api/auth', authRoutes);
33 app.use('/api', catalogRoutes);
34 app.use('/api', orderRoutes);
35 app.use('/api/notificacoes', notificationRoutes);
36 app.use('/api/relatorios', reportRoutes);
37
38 app.use(errorHandler);
39
40 const PORT = process.env.PORT || 3000;
41 app.listen(PORT, () => {
42   console.log('Server is running on port ${PORT}');
43 });
```

Listing 2: src/server.js

Arquivo: src/config/db.js

```
1 require('dotenv').config();
2 const { Pool } = require('pg');
3
4 const pool = new Pool({
5   host: process.env.DB_HOST,
6   port: process.env.DB_PORT,
7   database: process.env.DB_NAME,
8   user: process.env.DB_USER,
9   password: process.env.DB_PASS,
10 });
11
12 module.exports = pool;
```

Listing 3: src/config/db.js**Arquivo: src/config/mailler.js**

```
1 // src/config/mailler.js
2 const nodemailer = require('nodemailer');
3
4 const transporter = nodemailer.createTransport({
5   host: process.env.SMTP_HOST,
6   port: process.env.SMTP_PORT,
7   auth: {
8     user: process.env.SMTP_USER,
9     pass: process.env.SMTP_PASS,
10   },
11 });
12
13 const enviarEmailNotificacao = async ({ destinatario, assunto,
14   texto, html }) => {
15   if (!process.env.SMTP_HOST) {
16     console.warn('[Mailer] SMTP não configurado. Disparo ignorado
17       com segurança.');
```

```
27     return true;
28   } catch (err) {
29     console.error('[Mailer Error]:', err.message);
30     return false;
31   }
32 };
33
34 module.exports = { enviarEmailNotificacao };
```

Listing 4: src/config/mailer.js

Arquivo: src/middlewares/async.middleware.js

```
1 const asyncHandler = (fn) => (req, res, next) => {
2   return Promise.resolve(fn(req, res, next)).catch(next);
3 };
4
5 module.exports = asyncHandler;
```

Listing 5: src/middlewares/async.middleware.js

Arquivo: src/middlewares/auth.middleware.js

```
1 // src/middlewares/auth.middleware.js
2 const jwt = require('jsonwebtoken');
3
4 const verifyToken = (req, res, next) => {
5   const authHeader = req.headers['authorization'];
6   const token = authHeader && authHeader.split(' ')[1];
7
8   if (!token) {
9     return res.status(401).json({
10       erro: true,
11       mensagem: 'Token inválido ou não fornecido',
```

```

12     codigo: 'UNAUTHORIZED'
13   });
14 }
15
16 try {
17   const decoded = jwt.verify(token, process.env.JWT_SECRET || '
    supersecret');
18   req.user = decoded;
19   next();
20 } catch (error) {
21   return res.status(401).json({
22     erro: true,
23     mensagem: 'Token inválido ou não fornecido',
24     codigo: 'UNAUTHORIZED'
25   });
26 }
27 };
28
29 const requireRole = (rolesPermitidas) => {
30   return (req, res, next) => {
31     if (!req.user || !rolesPermitidas.includes(req.user.perfil)) {
32       return res.status(403).json({
33         erro: true,
34         mensagem: 'Acesso não autorizado para este perfil',
35         codigo: 'FORBIDDEN'
36       });
37     }
38     next();
39   };
40 };
41
42 module.exports = { verifyToken, requireRole };

```

Listing 6: src/middlewares/auth.middleware.js

Arquivo: src/middlewares/error.middleware.js

```
1 const errorHandler = (err, req, res, next) => {
2   const statusCode = err.statusCode || 500;
3   res.status(statusCode).json({
4     erro: true,
5     mensagem: err.message || 'Erro interno do servidor',
6     codigo: err.code || 'INTERNAL_SERVER_ERROR'
7   });
8 };
9
10 module.exports = errorHandler;
```

Listing 7: src/middlewares/error.middleware.js

Arquivo: src/modules/auth/auth.controller.js

```
1 // src/modules/auth/auth.controller.js
2 const authService = require('../auth.service');
3 const asyncHandler = require('../../middlewares/async.middleware');
4
5 class AuthController {
6   cadastrar = asyncHandler(async (req, res) => {
7     const novoUsuario = await authService.cadastrar(req.body);
8     res.status(201).json(novoUsuario);
9   });
10
11   login = asyncHandler(async (req, res) => {
12     const credenciais = await authService.login(req.body);
13     res.status(200).json(credenciais);
14   });
15 }
16
17 module.exports = new AuthController();
```

Listing 8: src/modules/auth/auth.controller.js

Arquivo: src/modules/auth/auth.repository.js

```
1 const pool = require('../../config/db');
2
3 class AuthRepository {
4   async buscarPorEmail(email) {
5     const query = `
6       SELECT id, nome, email, senha_hash, telefone, perfil
7       FROM usuarios
8       WHERE email = $1
9     `;
10    const result = await pool.query(query, [email]);
11    return result.rows[0];
12  }
13
14  async criarUsuario({ nome, email, senha_hash, telefone, perfil })
15  {
16    const query = `
17      INSERT INTO usuarios (nome, email, senha_hash, telefone,
18        perfil)
19      VALUES ($1, $2, $3, $4, $5)
20      RETURNING id, nome, email, telefone, perfil
21    `;
22    const values = [nome, email, senha_hash, telefone, perfil];
23    const result = await pool.query(query, values);
24    return result.rows[0];
25  }
26
27 module.exports = new AuthRepository();
```

Listing 9: src/modules/auth/auth.repository.js

Arquivo: src/modules/auth/auth.routes.js

```
1 // src/modules/auth/auth.routes.js
2 const express = require('express');
3 const authController = require('./auth.controller');
4
5 const router = express.Router();
6
7 router.post('/cadastro', authController.cadastrar);
8 router.post('/login', authController.login);
9
10 module.exports = router;
```

Listing 10: src/modules/auth/auth.routes.js

Arquivo: src/modules/auth/auth.service.js

```
1 const bcrypt = require('bcrypt');
2 const jwt = require('jsonwebtoken');
3 const authRepository = require('./auth.repository');
4
5 class AuthService {
6   async cadastrar({ nome, email, senha, telefone, perfil }) {
7     if (!nome || !email || !senha || !perfil) {
8       const error = new Error('Campos obrigatórios: nome, email,
9         senha e perfil');
10       error.statusCode = 400;
11       throw error;
12     }
13
14     const perfisValidos = ['CLIENTE', 'PRODUTOR', 'ORGANIZACAO'];
```

```
14     if (!perfisValidos.includes(perfil)) {
15         const error = new Error('Perfil inválido');
16         error.statusCode = 400;
17         throw error;
18     }
19
20     if (perfil === 'PRODUTOR' && !telefone) {
21         const error = new Error('Telefone é obrigatório para
22             produtores');
23         error.statusCode = 400;
24         throw error;
25     }
26
27     const usuarioExistente = await authRepository.buscarPorEmail(
28         email);
29
30     if (usuarioExistente) {
31         const error = new Error('E-mail já cadastrado');
32         error.statusCode = 409;
33         throw error;
34     }
35
36     const salt = await bcrypt.genSalt(10);
37     const senha_hash = await bcrypt.hash(senha, salt);
38
39     const novoUsuario = await authRepository.criarUsuario({
40         nome,
41         email,
42         senha_hash,
43         telefone: telefone || null,
44         perfil
45     });
46
47     return novoUsuario;
48 }
```

```
46
47 async login({ email, senha }) {
48   if (!email || !senha) {
49     const error = new Error('Email e senha são obrigatórios');
50     error.statusCode = 400;
51     throw error;
52   }
53
54   const usuario = await authRepository.buscarPorEmail(email);
55   if (!usuario) {
56     const error = new Error('Credenciais inválidas');
57     error.statusCode = 401;
58     throw error;
59   }
60
61   const isMatch = await bcrypt.compare(senha, usuario.senha_hash)
62   ;
63   if (!isMatch) {
64     const error = new Error('Credenciais inválidas');
65     error.statusCode = 401;
66     throw error;
67   }
68
69   const payload = {
70     id: usuario.id,
71     nome: usuario.nome,
72     perfil: usuario.perfil
73   };
74
75   const token = jwt.sign(
76     payload,
77     process.env.JWT_SECRET || 'supersecret',
78     { expiresIn: process.env.JWT_EXPIRES_IN || '8h' }
79   );
```

```
79
80     return {
81         token,
82         usuario: payload
83     };
84 }
85 }
86
87 module.exports = new AuthService();
```

Listing 11: src/modules/auth/auth.service.js

Arquivo: src/modules/catalog/catalog.controller.js

```
1 // src/modules/catalog/catalog.controller.js
2 const catalogService = require('../catalog.service');
3 const asyncHandler = require('../../middlewares/async.middleware');
4
5 class CatalogController {
6     listarCatalogo = asyncHandler(async (req, res) => {
7         const produtos = await catalogService.listarCatalogo();
8         res.status(200).json(produtos);
9     });
10
11     listarProdutosDoProdutor = asyncHandler(async (req, res) => {
12         const produtos = await catalogService.listarProdutosDoProdutor(
13             req.user.id);
14         res.status(200).json(produtos);
15     });
16
17     criarProduto = asyncHandler(async (req, res) => {
18         const produto = await catalogService.criarProduto(req.user.id,
19             req.body);
20         res.status(201).json(produto);
21     });
22 }
```

```

19     });
20
21     atualizarProduto = asyncHandler(async (req, res) => {
22         const { id } = req.params;
23         const produto = await catalogService.atualizarProduto(id, req.
            user.id, req.body);
24         res.status(200).json(produto);
25     });
26
27     desativarProduto = asyncHandler(async (req, res) => {
28         const { id } = req.params;
29         const resultado = await catalogService.desativarProduto(id, req
            .user.id);
30         res.status(200).json(resultado);
31     });
32 }
33
34 module.exports = new CatalogController();

```

Listing 12: src/modules/catalog/catalog.controller.js

Arquivo: src/modules/catalog/catalog.repository.js

```

1 // src/modules/catalog/catalog.repository.js
2 const pool = require('.../.../config/db');
3
4 class CatalogRepository {
5     async listarCatalogoPublico() {
6         const query = `
7             SELECT p.id, p.nome, p.descricao, p.preco, p.unidade_medida,
8                 u.nome AS produtor_nome, u.id AS produtor_id
9             FROM produtos p
10             INNER JOIN usuarios u ON p.produtor_id = u.id
11             WHERE p.ativo = TRUE

```

```
12         ORDER BY p.nome ASC
13     ';
14     const result = await pool.query(query);
15     return result.rows;
16 }
17
18 async listarPorProdutor(produtorId) {
19     const query = '
20         SELECT id, nome, descricao, preco, unidade_medida, ativo
21         FROM produtos
22         WHERE produtor_id = $1 AND ativo = TRUE
23         ORDER BY nome ASC
24     ';
25     const result = await pool.query(query, [produtorId]);
26     return result.rows;
27 }
28
29 async buscarPorId(id) {
30     const query = 'SELECT * FROM produtos WHERE id = $1';
31     const result = await pool.query(query, [id]);
32     return result.rows[0];
33 }
34
35 async criarProduto({ produtor_id, nome, descricao, preco,
36     unidade_medida }) {
37     const query = '
38         INSERT INTO produtos (produtor_id, nome, descricao, preco,
39             unidade_medida)
40         VALUES ($1, $2, $3, $4, $5)
41         RETURNING *
42     ';
43     const values = [produtor_id, nome, descricao, preco,
44         unidade_medida];
45     const result = await pool.query(query, values);
```



```
43     return result.rows[0];
44 }
45
46 async atualizarProduto(id, produtorId, { nome, descricao, preco,
47     unidade_medida }) {
48     const query = `
49         UPDATE produtos
50         SET nome = $1, descricao = $2, preco = $3, unidade_medida =
51             $4
52         WHERE id = $5 AND produtor_id = $6
53         RETURNING *
54     `;
55     const values = [nome, descricao, preco, unidade_medida, id,
56         produtorId];
57     const result = await pool.query(query, values);
58     return result.rows[0];
59 }
60
61 async desativarProduto(id, produtorId) {
62     const query = `
63         UPDATE produtos
64         SET ativo = FALSE
65         WHERE id = $1 AND produtor_id = $2
66         RETURNING id
67     `;
68     const result = await pool.query(query, [id, produtorId]);
69     return result.rows[0];
70 }
71
72 module.exports = new CatalogRepository();
```

Listing 13: src/modules/catalog/catalog.repository.js

Arquivo: src/modules/catalog/catalog.routes.js

```
1 // src/modules/catalog/catalog.routes.js
2 const express = require('express');
3 const catalogController = require('./catalog.controller');
4 const { verifyToken, requireRole } = require('../middlewares/
    auth.middleware');
5
6 const router = express.Router();
7
8 router.get('/catalogo', catalogController.listarCatalogo);
9 router.get('/produtos/produtor', verifyToken, requireRole(['
    PRODUTOR']), catalogController.listarProdutosDoProdutor);
10 router.post('/produtos', verifyToken, requireRole(['PRODUTOR']),
    catalogController.criarProduto);
11 router.put('/produtos/:id', verifyToken, requireRole(['PRODUTOR']),
    catalogController.atualizarProduto);
12 router.delete('/produtos/:id', verifyToken, requireRole(['PRODUTOR'
    ]), catalogController.desativarProduto);
13
14 module.exports = router;
```

Listing 14: src/modules/catalog/catalog.routes.js

Arquivo: src/modules/catalog/catalog.service.js

```
1 // src/modules/catalog/catalog.service.js
2 const catalogRepository = require('./catalog.repository');
3
4 class CatalogService {
5   async listarCatalogo() {
6     const produtos = await catalogRepository.listarCatalogoPublico
7       ();
8     return produtos.map(p => ({
9       ...p,
```

```
9         preco: Number(p.preco)
10     }));
11 }
12
13 async listarProdutosDoProdutor(produtorId) {
14     const produtos = await catalogRepository.listarPorProdutor(
15         produtorId);
16     return produtos.map(p => ({
17         ...p,
18         preco: Number(p.preco)
19     }));
20 }
21
22 async criarProduto(produtorId, dados) {
23     const { nome, descricao, preco, unidade_medida } = dados;
24
25     if (!nome || preco === undefined || !unidade_medida) {
26         const error = new Error('Campos obrigatórios: nome, preco e
27             unidade_medida');
28         error.statusCode = 400;
29         throw error;
30     }
31
32     if (Number(preco) < 0) {
33         const error = new Error('0 preço não pode ser negativo');
34         error.statusCode = 400;
35         throw error;
36     }
37
38     return await catalogRepository.criarProduto({
39         produtor_id: produtorId,
40         nome,
41         descricao: descricao || null,
42         preco,
```

```
41     unidade_medida
42   });
43 }
44
45 async atualizarProduto(id, produtorId, dados) {
46   const produtoExistente = await catalogRepository.buscarPorId(id
47   );
48
49   if (!produtoExistente) {
50     const error = new Error('Produto não encontrado');
51     error.statusCode = 404;
52     throw error;
53   }
54
55   if (produtoExistente.produtor_id !== produtorId) {
56     const error = new Error('Acesso negado para modificar este
57     produto');
58     error.statusCode = 403;
59     throw error;
60   }
61
62   const { nome, descricao, preco, unidade_medida } = dados;
63
64   return await catalogRepository.atualizarProduto(id, produtorId,
65   {
66     nome: nome || produtoExistente.nome,
67     descricao: descricao !== undefined ? descricao :
68     produtoExistente.descricao,
69     preco: preco !== undefined ? preco : produtoExistente.preco,
70     unidade_medida: unidade_medida || produtoExistente.
71     unidade_medida
72   });
73 }
```

```

70  async desativarProduto(id, produtorId) {
71      const produtoExistente = await catalogRepository.buscarPorId(id
72          );
73
74      if (!produtoExistente) {
75          const error = new Error('Produto não encontrado');
76          error.statusCode = 404;
77          throw error;
78      }
79
80      if (produtoExistente.produtor_id !== produtorId) {
81          const error = new Error('Acesso negado para desativar este
82              produto');
83          error.statusCode = 403;
84          throw error;
85      }
86
87      await catalogRepository.desativarProduto(id, produtorId);
88      return { mensagem: 'Produto desativado com sucesso' };
89  }
90  module.exports = new CatalogService();

```

Listing 15: src/modules/catalog/catalog.service.js

Arquivo: src/modules/notification/notification.controller.js

```

1  // src/modules/notification/notification.controller.js
2  const notificationService = require('../notification.service');
3  const asyncHandler = require('../../middlewares/async.middleware');
4
5  class NotificationController {
6      listar = asyncHandler(async (req, res) => {

```

```

7      const notificacoes = await notificationService.
          listarNotificacoes(req.user.id);
8      res.status(200).json(notificacoes);
9  });
10
11  marcarComoLida = asyncHandler(async (req, res) => {
12      const { id } = req.params;
13      const produtorId = req.user.id;
14      const resultado = await notificationService.marcarLida(id,
          produtorId);
15      res.status(200).json(resultado);
16  });
17  }
18
19  module.exports = new NotificationController();

```

Listing 16: src/modules/notification/notification.controller.js

Arquivo: src/modules/notification/notification.repository.js

```

1  // src/modules/notification/notification.repository.js
2  const pool = require('../../config/db');
3
4  class NotificationRepository {
5      async listarPorProdutor(produtorId) {
6          const query = `
7              SELECT n.id, n.pedido_id, n.mensagem, n.lida,
8                     p.comprovante_codigo, p.criado_em AS data_pedido
9              FROM notificacoes n
10             JOIN pedidos p ON n.pedido_id = p.id
11             WHERE n.produtor_id = $1
12             ORDER BY n.lida ASC, p.criado_em DESC;
13          `;
14          const result = await pool.query(query, [produtorId]);

```

```

15     return result.rows;
16 }
17
18 async marcarComoLida(notificacaoId, produtorId) {
19     const query = '
20         UPDATE notificacoes
21         SET lida = TRUE
22         WHERE id = $1 AND produtor_id = $2
23         RETURNING id, lida;
24     ';
25     const result = await pool.query(query, [notificacaoId,
26         produtorId]);
27     return result.rows[0];
28 }
29
30 module.exports = new NotificationRepository();

```

Listing 17: src/modules/notification/notification.repository.js

Arquivo: src/modules/notification/notification.routes.js

```

1 // src/modules/notification/notification.routes.js
2 const express = require('express');
3 const notificationController = require('./notification.controller')
4     ;
5
6 const { verifyToken, requireRole } = require('../middlewares/
7     auth.middleware');
8
9 const router = express.Router();
10
11 router.get('/produtor', verifyToken, requireRole(['PRODUTOR']),
12     notificationController.listar);

```

```
9 router.patch('/:id/lida', verifyToken, requireRole(['PRODUTOR']),
    notificationController.marcarComoLida);
10
11 module.exports = router;
```

Listing 18: src/modules/notification/notification.routes.js

Arquivo: src/modules/notification/notification.service.js

```
1 // src/modules/notification/notification.service.js
2 const notificationRepository = require('./notification.repository')
    ;
3
4 class NotificationService {
5     async listarNotificacoes(produtorId) {
6         return await notificationRepository.listarPorProdutor(
            produtorId);
7     }
8
9     async marcarLida(notificacaoId, produtorId) {
10         const result = await notificationRepository.marcarComoLida(
            notificacaoId, produtorId);
11
12         if (!result) {
13             const error = new Error('Notificação não encontrada ou não
                autorizada');
14             error.statusCode = 404;
15             throw error;
16         }
17
18         return { mensagem: 'Notificação marcada como lida com sucesso'
            };
19     }
20 }
```



```

21
22 module.exports = new NotificationService();

```

Listing 19: src/modules/notification/notification.service.js

Arquivo: src/modules/order/order.controller.js

```

1 // src/modules/order/order.controller.js
2 const orderService = require('../order.service');
3 const asyncHandler = require('../../middlewares/async.middleware');
4
5 class OrderController {
6   listarPontosRetirada = asyncHandler(async (req, res) => {
7     const pontos = await orderService.listarPontosRetirada();
8     res.status(200).json(pontos);
9   });
10
11   fecharPedido = asyncHandler(async (req, res) => {
12     const clienteId = req.user.id;
13     const pedido = await orderService.fecharPedido(clienteId, req.
14       body);
15     res.status(201).json(pedido);
16   });
17
18   obterComprovante = asyncHandler(async (req, res) => {
19     const { id } = req.params;
20     const usuarioId = req.user.id;
21     const perfil = req.user.perfil;
22     const comprovante = await orderService.obterComprovante(id,
23       usuarioId, perfil);
24     res.status(200).json(comprovante);
25   });
26
27   listarHistorico = asyncHandler(async (req, res) => {

```

```

26     const historico = await orderService.listarHistoricoCliente(req
      .user.id);
27     res.status(200).json(historico);
28   });
29
30   listarPedidosProdutor = asyncHandler(async (req, res) => {
31     const pedidos = await orderService.listarPedidosProdutor(req.
      user.id);
32     res.status(200).json(pedidos);
33   });
34
35   atualizarStatus = asyncHandler(async (req, res) => {
36     const { id } = req.params;
37     const produtorId = req.user.id;
38     const { status } = req.body;
39     const resultado = await orderService.atualizarStatusPedido(id,
      produtorId, status);
40     res.status(200).json(resultado);
41   });
42 }
43
44 module.exports = new OrderController();

```

Listing 20: src/modules/order/order.controller.js

Arquivo: src/modules/order/order.repository.js

```

1 // src/modules/order/order.repository.js
2 const pool = require('../../config/db');
3
4 class OrderRepository {
5   async listarPontosRetirada() {
6     const query = 'SELECT id, nome, endereco FROM pontos_retirada
      ORDER BY nome ASC';

```

```

7     const result = await pool.query(query);
8     return result.rows;
9 }
10
11 async buscarEdicaoAtiva() {
12     const query = 'SELECT id, nome_identificador FROM edicoes_feira
13         WHERE ativa = TRUE LIMIT 1';
14     const result = await pool.query(query);
15     return result.rows[0] || null;
16 }
17
18 async buscarPontoRetiradaPorId(id) {
19     const query = 'SELECT id, nome, endereco FROM pontos_retirada
20         WHERE id = $1';
21     const result = await pool.query(query, [id]);
22     return result.rows[0] || null;
23 }
24
25 async buscarProdutosPorIds(ids) {
26     const query = '
27         SELECT id, produtor_id, nome, preco, ativo
28         FROM produtos
29         WHERE id = ANY($1::uuid[]) AND ativo = TRUE
30     ';
31     const result = await pool.query(query, [ids]);
32     return result.rows;
33 }
34
35 async criarPedidoTransacional({ cliente_id, edicao_id,
36     ponto_retirada_id, comprovante_codigo, status, valor_total,
37     itens, notificacoes }) {
38     const client = await pool.connect();
39     try {
40         await client.query('BEGIN');

```

```
37
38     const insertPedidoQuery = `
39         INSERT INTO pedidos (cliente_id, edicao_id,
40             ponto_retirada_id, comprovante_codigo, status,
41             valor_total)
42         VALUES ($1, $2, $3, $4, $5, $6)
43         RETURNING id, comprovante_codigo, status, valor_total,
44             criado_em;
45     `;
46     const pedidoRes = await client.query(insertPedidoQuery, [
47         cliente_id, edicao_id, ponto_retirada_id,
48         comprovante_codigo, status, valor_total
49     ]);
50     const pedido = pedidoRes.rows[0];
51
52     const insertItemQuery = `
53         INSERT INTO itens_pedido (pedido_id, produto_id, quantidade
54             , preco_unitario)
55         VALUES ($1, $2, $3, $4);
56     `;
57     for (const item of itens) {
58         await client.query(insertItemQuery, [
59             pedido.id, item.produto_id, item.quantidade, item.
60                 preco_unitario
61         ]);
62     }
63
64     const insertNotificacaoQuery = `
65         INSERT INTO notificacoes (produtor_id, pedido_id, mensagem)
66         VALUES ($1, $2, $3);
67     `;
68     for (const notif of notificacoes) {
69         await client.query(insertNotificacaoQuery, [
70             notif.produtor_id, pedido.id, notif.mensagem
71         ]);
72     }
73 }
```

```

65         });
66     }
67
68     await client.query('COMMIT');
69     return pedido;
70 } catch (error) {
71     await client.query('ROLLBACK');
72     throw error;
73 } finally {
74     client.release();
75 }
76 }
77
78 async buscarComprovantePorId(pedidoId, usuarioId, perfil) {
79     const pedidoQuery = '
80         SELECT p.id, p.comprovante_codigo, p.status, p.valor_total, p
81             .criado_em, p.cliente_id,
82             pr.nome AS ponto_nome, pr.endereco AS ponto_endereco,
83             ef.nome_identificador AS edicao_nome,
84             u_cli.nome AS cliente_nome, u_cli.telefone AS
85                 cliente_telefone
86
87         FROM pedidos p
88         JOIN pontos_retirada pr ON p.ponto_retirada_id = pr.id
89         JOIN edicoes_feira ef ON p.edicao_id = ef.id
90         JOIN usuarios u_cli ON p.cliente_id = u_cli.id
91         WHERE p.id = $1;
92
93     ';
94
95     const pedidoRes = await pool.query(pedidoQuery, [pedidoId]);
96     const pedido = pedidoRes.rows[0];
97
98     if (!pedido) return null;
99
100     if (perfil === 'CLIENTE' && pedido.cliente_id !== usuarioId) {
101         const error = new Error('Acesso não autorizado ao pedido');

```

```

97     error.statusCode = 403;
98     error.codigo = 'FORBIDDEN';
99     throw error;
100 }
101
102 const itensQuery = '
103     SELECT ip.quantidade, ip.preco_unitario, prod.nome AS
104         produto_nome,
105         u_prod.id AS produtor_id, u_prod.nome AS produtor_nome
106         , u_prod.telefone AS produtor_telefone
107     FROM itens_pedido ip
108     JOIN produtos prod ON ip.produto_id = prod.id
109     JOIN usuarios u_prod ON prod.produtor_id = u_prod.id
110     WHERE ip.pedido_id = $1;
111 ';
112
113 const itensRes = await pool.query(itensQuery, [pedidoId]);
114 pedido.itens = itensRes.rows;
115
116 return pedido;
117 }
118
119 async listarPorCliente(clienteId) {
120     const query = '
121         SELECT p.id, p.comprovante_codigo, p.status, p.valor_total, p
122             .criado_em,
123             pr.nome AS ponto_nome, pr.endereco AS ponto_endereco
124     FROM pedidos p
125     JOIN pontos_retirada pr ON p.ponto_retirada_id = pr.id
126     WHERE p.cliente_id = $1
127     ORDER BY p.criado_em DESC;
128 ';
129     const result = await pool.query(query, [clienteId]);
130     return result.rows;
131 }

```

```

128
129 async listarPorProdutor(produtorId) {
130     const query = '
131         SELECT DISTINCT p.id, p.comprovante_codigo, p.status, p.
132             valor_total, p.criado_em,
133             pr.nome AS ponto_nome,
134             u_cli.nome AS cliente_nome, u_cli.telefone AS
135                 cliente_telefone
136
137     FROM pedidos p
138     JOIN pontos_retirada pr ON p.ponto_retirada_id = pr.id
139     JOIN usuarios u_cli ON p.cliente_id = u_cli.id
140     JOIN itens_pedido ip ON p.id = ip.pedido_id
141     JOIN produtos prod ON ip.produto_id = prod.id
142     WHERE prod.produtor_id = $1
143     ORDER BY p.criado_em DESC;
144     ';
145     const result = await pool.query(query, [produtorId]);
146     return result.rows;
147 }
148
149 async buscarItensDoProdutorNoPedido(pedidoId, produtorId) {
150     const query = '
151         SELECT ip.quantidade, ip.preco_unitario, prod.nome AS
152             produto_nome
153
154     FROM itens_pedido ip
155     JOIN produtos prod ON ip.produto_id = prod.id
156     WHERE ip.pedido_id = $1 AND prod.produtor_id = $2;
157     ';
158     const result = await pool.query(query, [pedidoId, produtorId]);
159     return result.rows;
160 }
161
162 async verificarProdutorPertenceAoPedido(pedidoId, produtorId) {
163     const query = '

```

```

159     SELECT 1 FROM itens_pedido ip
160     JOIN produtos prod ON ip.produto_id = prod.id
161     WHERE ip.pedido_id = $1 AND prod.produtor_id = $2
162     LIMIT 1;
163     ';
164     const result = await pool.query(query, [pedidoId, produtorId]);
165     return result.rows.length > 0;
166 }
167
168 async buscarStatusAtual(pedidoId) {
169     const query = 'SELECT id, status, comprovante_codigo FROM
170     pedidos WHERE id = $1;';
171     const result = await pool.query(query, [pedidoId]);
172     return result.rows[0];
173 }
174
175 async atualizarStatus(pedidoId, novoStatus) {
176     const query = '
177     UPDATE pedidos
178     SET status = $1
179     WHERE id = $2
180     RETURNING id, status, comprovante_codigo;
181     ';
182     const result = await pool.query(query, [novoStatus, pedidoId]);
183     return result.rows[0];
184 }
185
186 module.exports = new OrderRepository();

```

Listing 21: src/modules/order/order.repository.js

Arquivo: src/modules/order/order.routes.js


```

1 // src/modules/order/order.routes.js
2 const express = require('express');
3 const orderController = require('./order.controller');
4 const { verifyToken, requireRole } = require('../middlewares/
    auth.middleware');
5
6 const router = express.Router();
7
8 router.get('/pontos-retirada', orderController.listarPontosRetirada
    );
9 router.post('/pedidos', verifyToken, requireRole(['CLIENTE']),
    orderController.fecharPedido);
10 router.get('/pedidos/historico', verifyToken, requireRole(['CLIENTE
    ']), orderController.listarHistorico);
11 router.get('/pedidos/produtor', verifyToken, requireRole(['PRODUTOR
    ']), orderController.listarPedidosProdutor);
12 router.get('/pedidos/:id/comprovante', verifyToken, orderController
    .obterComprovante);
13 router.patch('/pedidos/:id/status', verifyToken, requireRole(['
    PRODUTOR']), orderController.atualizarStatus);
14
15 module.exports = router;

```

Listing 22: src/modules/order/order.routes.js

Arquivo: src/modules/order/order.service.js

```

1 // src/modules/order/order.service.js
2 const orderRepository = require('./order.repository');
3
4 class OrderService {
5   async listarPontosRetirada() {
6     return await orderRepository.listarPontosRetirada();
7   }

```

```
8
9  async fecharPedido(clienteId, { ponto_retirada_id, itens }) {
10    if (!ponto_retirada_id) {
11      const error = new Error('Ponto de retirada é obrigatório');
12      error.statusCode = 400;
13      throw error;
14    }
15
16    if (!Array.isArray(itens) || itens.length === 0) {
17      const error = new Error('0 carrinho não pode estar vazio');
18      error.statusCode = 400;
19      throw error;
20    }
21
22    const ponto = await orderRepository.buscarPontoRetiradaPorId(
23      ponto_retirada_id);
24    if (!ponto) {
25      const error = new Error('Ponto de retirada inválido');
26      error.statusCode = 404;
27      throw error;
28    }
29
30    const edicao = await orderRepository.buscarEdicaoAtiva();
31    if (!edicao) {
32      const error = new Error('Não há nenhuma edição de feira ativa
33        no momento');
34      error.statusCode = 400;
35      throw error;
36    }
37
38    const idsProdutos = itens.map(i => i.produto_id);
39    const dbProdutos = await orderRepository.buscarProdutosPorIds(
40      idsProdutos);
```

```
39   if (dbProdutos.length !== idsProdutos.length) {
40       const error = new Error('Um ou mais produtos não estão mais
        disponíveis');
41       error.statusCode = 400;
42       throw error;
43   }
44
45   let valor_total = 0;
46   const itensMapeados = [];
47   const produtoresSet = new Set();
48
49   for (const reqItem of itens) {
50       if (reqItem.quantidade <= 0) {
51           const error = new Error('Quantidade inválida para o produto
                ' + reqItem.produto_id);
52           error.statusCode = 400;
53           throw error;
54       }
55
56       const dbProd = dbProdutos.find(p => p.id === reqItem.
            produto_id);
57       const precoUnitario = Number(dbProd.preco);
58       valor_total += precoUnitario * reqItem.quantidade;
59
60       itensMapeados.push({
61           produto_id: reqItem.produto_id,
62           quantidade: reqItem.quantidade,
63           preco_unitario: precoUnitario
64       });
65
66       produtoresSet.add(dbProd.produtor_id);
67   }
68
```

```
69     const comprovante_codigo = 'SISF-' + Math.floor(1000 + Math.  
70         random() * 9000);  
71  
72     const notificacoes = Array.from(produtoresSet).map(produtor_id  
73         => ({  
74             produtor_id,  
75             mensagem: 'Você tem um novo pedido com itens a serem  
76                 preparados'  
77         }));  
78  
79     const pedido = await orderRepository.criarPedidoTransacional({  
80         cliente_id: clienteId,  
81         edicao_id: edicao.id,  
82         ponto_retirada_id,  
83         comprovante_codigo,  
84         status,  
85         valor_total,  
86         itens: itensMapeados,  
87         notificacoes  
88     });  
89  
90     return pedido;  
91 }  
92  
93 async obterComprovante(pedidoId, usuarioId, perfil) {  
94     const comprovante = await orderRepository.  
95         buscarComprovantePorId(pedidoId, usuarioId, perfil);  
96     if (!comprovante) {  
97         const error = new Error('Pedido não encontrado');  
98         error.statusCode = 404;  
99         throw error;  
100     }  
101 }
```

```
109     comprovante.valor_total = Number(comprovante.valor_total);
110     comprovante.itens = comprovante.itens.map(item => ({
111         ...item,
112         preco_unitario: Number(item.preco_unitario)
113     }));
114
115     return comprovante;
116 }
117
118 async listarHistoricoCliente(clienteId) {
119     const historico = await orderRepository.listarPorCliente(
120         clienteId);
121     return historico.map(p => ({
122         ...p,
123         valor_total: Number(p.valor_total)
124     }));
125 }
126
127 async listarPedidosProdutor(produtorId) {
128     const pedidos = await orderRepository.listarPorProdutor(
129         produtorId);
130
131     for (const pedido of pedidos) {
132         pedido.valor_total = Number(pedido.valor_total);
133         pedido.itens = await orderRepository.
134             buscarItensDoProdutorNoPedido(pedido.id, produtorId);
135     }
136
137     return pedidos;
138 }
139
140 async atualizarStatusPedido(pedidoId, produtorId, novoStatus) {
141     const statusValidos = ['RECEBIDO', 'EM_PREPARACAO', 'ENTREGUE',
142         ''];
143 }
```

```
129     if (!statusValidos.includes(novoStatus)) {
130         const error = new Error('Status inválido');
131         error.statusCode = 400;
132         throw error;
133     }
134
135     const pedido = await orderRepository.buscarStatusAtual(pedidoId
136         );
137     if (!pedido) {
138         const error = new Error('Pedido não encontrado');
139         error.statusCode = 404;
140         throw error;
141     }
142
143     const pertence = await orderRepository.
144         verificarProdutorPertenceAoPedido(pedidoId, produtorId);
145     if (!pertence) {
146         const error = new Error('Acesso não autorizado para alterar
147             este pedido');
148         error.statusCode = 403;
149         throw error;
150     }
151
152     const statusAtual = pedido.status;
153     if (statusAtual === 'RECEBIDO' && novoStatus !== 'EM_PREPARACAO
154         ') {
155         const error = new Error('Transição inválida: pedidos
156             recebidos só podem avançar para EM_PREPARACAO');
157         error.statusCode = 400;
158         throw error;
159     }
160
161     if (statusAtual === 'EM_PREPARACAO' && novoStatus !== 'ENTREGUE
162         ') {
```

```

156     const error = new Error('Transição inválida: pedidos em
157         preparação só podem avançar para ENTREGUE');
158     error.statusCode = 400;
159     throw error;
160 }
161 if (statusAtual === 'ENTREGUE') {
162     const error = new Error('Transição inválida: pedido já se
163         encontra finalizado como ENTREGUE');
164     error.statusCode = 400;
165     throw error;
166 }
167
168 const atualizado = await orderRepository.atualizarStatus(
169     pedidoId, novoStatus);
170
171 return {
172     sucesso: true,
173     mensagem: 'Status do pedido alterado para ${novoStatus} com
174         sucesso.',
175     status: atualizado.status
176 };
177 }
178 }
179
180 module.exports = new OrderService();

```

Listing 23: src/modules/order/order.service.js

Arquivo: src/modules/report/report.controller.js

```

1 // src/modules/report/report.controller.js
2 const reportService = require('../report.service');
3 const asyncHandler = require('.../middlewares/async.middleware');
4

```

```

5 class ReportController {
6   obterRelatorioVendas = asyncHandler(async (req, res) => {
7     const relatorio = await reportService.gerarRelatorioVendas(req.
        user.id);
8     res.status(200).json(relatorio);
9   });
10 }
11
12 module.exports = new ReportController();

```

Listing 24: src/modules/report/report.controller.js

Arquivo: src/modules/report/report.repository.js

```

1 // src/modules/report/report.repository.js
2 const pool = require('../../config/db');
3
4 class ReportRepository {
5   async obterMetricasGerais(produtorId) {
6     const query = `
7       SELECT
8         COALESCE(SUM(ip.quantidade * ip.preco_unitario), 0)::
          numeric(10,2) AS total_faturado,
9         COUNT(DISTINCT ip.pedido_id)::int AS total_pedidos
10      FROM itens_pedido ip
11      JOIN produtos p ON ip.produto_id = p.id
12      WHERE p.produtor_id = $1;
13     `;
14     const result = await pool.query(query, [produtorId]);
15     return result.rows[0];
16   }
17
18   async obterItensMaisVendidos(produtorId) {
19     const query = `

```



```

20     SELECT
21         p.nome AS produto_nome ,
22         SUM(ip.quantidade)::int AS quantidade_vendida ,
23         SUM(ip.quantidade * ip.preco_unitario)::numeric(10,2) AS
            subtotal
24     FROM itens_pedido ip
25     JOIN produtos p ON ip.produto_id = p.id
26     WHERE p.produtor_id = $1
27     GROUP BY p.nome
28     ORDER BY quantidade_vendida DESC;
29     ';
30     const result = await pool.query(query, [produtorId]);
31     return result.rows;
32 }
33 }
34
35 module.exports = new ReportRepository();

```

Listing 25: src/modules/report/report.repository.js

Arquivo: src/modules/report/report.routes.js

```

1 // src/modules/report/report.routes.js
2 const express = require('express');
3 const reportController = require('./report.controller');
4 const { verifyToken, requireRole } = require('../../middlewares/
    auth.middleware');
5
6 const router = express.Router();
7
8 router.get('/vendas/produtor', verifyToken, requireRole(['PRODUTOR',
    ]), reportController.obterRelatorioVendas);
9
10 module.exports = router;

```

Listing 26: src/modules/report/report.routes.js

Arquivo: src/modules/report/report.service.js

```
1 // src/modules/report/report.service.js
2 const reportRepository = require('../report.repository');
3
4 class ReportService {
5   async gerarRelatorioVendas(produtorId) {
6     const [metricas, itens] = await Promise.all([
7       reportRepository.obterMetricasGerais(produtorId),
8       reportRepository.obterItensMaisVendidos(produtorId)
9     ]);
10
11     return {
12       total_faturado: parseFloat(metricas.total_faturado) || 0,
13       total_pedidos: parseInt(metricas.total_pedidos, 10) || 0,
14       itens_mais_vendidos: itens.map(item => ({
15         produto_nome: item.produto_nome,
16         quantidade_vendida: parseInt(item.quantidade_vendida, 10),
17         subtotal: parseFloat(item.subtotal)
18       })))
19     };
20   }
21 }
22
23 module.exports = new ReportService();
```

Listing 27: src/modules/report/report.service.js

APÊNDICE C - REGISTRO DE OBSERVAÇÕES DE ATIVIDADE PRÁTICA

DADOS DOS ESTUDANTES

Estudante 1: _____	Turma: _____
Estudante 2: _____	Turma: _____
Estudante 3: _____	Turma: _____
Unidade Escolar/Município: _____	Curso: _____
Componente Curricular: _____	
Instrutor Responsável: _____	

LOCAL DE REALIZAÇÃO DA ATIVIDADE PRÁTICA

Local: _____
Setor envolvido (visitado/observado): _____

REGISTRO DOS PROCEDIMENTOS/FATOS OBSERVADOS

Data da Observação: ____ / ____ / ____ Horário: de ____ h às ____ h.

Registros:

Data: ____ / ____ / ____ .	Data: ____ / ____ / ____ .
1. _____	1. _____
2. _____	
3. _____	

Assinatura dos Estudantes

Assinatura do Instrutor.

Observação: Os registros de observações servirão para a elaboração do Relatório Técnico ou do Projeto Técnico.